

AI / ML ENGINEER SYLLABUS

A Beginner-Friendly, Step-by-Step Study Guide (Zero to Job-Ready)

Who this is for: Complete beginners with no prior AI/ML background. Every topic is explained in plain language before you dive in — no assumed knowledge.

How this guide is organized — every week follows the same easy-to-follow format:

WHY	Why this matters — a plain-language reason you're learning this, before the jargon starts
KEY TERMS	Key terms explained — new vocabulary defined simply, so nothing feels confusing
WEEKLY	Weekly checklist — small tickable topics, so you always know exactly what's next
BUILD	Build / Practice — a small hands-on task so you learn by doing, not just reading
TIP	Beginner tips — advice to avoid common early mistakes
CHECKPT	Checkpoints — a simple self-test before you allow yourself to move to the next phase

Total length	52 weeks (about 12 months) at a relaxed, sustainable pace
Weekly routine	5 focused study days + 1 project/practice day + 1 light review day
Golden rule	Learn → Code it yourself → Practice → Build a small project → Review. Never skip straight to "build".
Main tools	Python, Jupyter/VS Code, Git & GitHub, and free/open datasets throughout

Before you start — 5 rules for beginners:

- Go in order. Later topics (LLMs, agents) genuinely require the earlier foundations (Python, math, ML basics).
- Type every line of code yourself. Copy-pasting feels productive but teaches you nothing.
- It's normal to feel confused in week 1 of any new topic. Confusion is part of learning, not a sign you're behind.
- Google and ask AI assistants for help — but always try to solve it yourself first for 15–20 minutes.
- Celebrate finishing small projects. They matter more than finishing chapters.

Your 52-Week Roadmap at a Glance

Stage	Weeks	What you'll be able to do after this stage
1. Python Basics	1-6	Write simple Python programs on your own - no tutorials needed
2. Python + Math Foundations	7-12	Understand OOP, use Git properly, and know the core math behind AI
3. Data Analysis	13-16	Clean messy data and find patterns/insights using Pandas and charts
4. SQL for Data	17-19	Query and analyze data stored in databases
5. Statistics and Probability	20-22	Understand and apply the stats concepts every ML model relies on
6. Machine Learning Basics	23-24	Explain what ML is and how a model actually learns
7. Regression and Classification	25-27	Build models that predict numbers or categories
8. Model Building and Tuning	28-31	Build a complete, tuned, evaluated ML project from scratch
9. Deep Learning and PyTorch	32-35	Build and train your own neural networks
10. NLP, CV and Transformers	36-39	Understand how modern AI (like ChatGPT) actually works
11. LLMs and GenAI APIs	40-42	Build real applications powered by AI models
12. RAG and Vector Databases	43-44	Make AI answer questions using your own private documents
13. AI Agents	45-46	Build AI that can use tools and complete multi-step tasks
14. Fine-Tuning and MLOps	47-48	Customize a model and track/monitor it responsibly
15. Deployment and System Design	49-50	Turn your AI project into a real, live web service
16. Capstone and Job Prep	51-52	Ship a portfolio project and prepare for interviews

TIP: If a stage takes you longer than planned, that's fine — the week numbers are a guide, not a deadline. Move on only when the checkpoint at the end of each stage feels easy.

PHASE 1

Python Basics

Weeks 1–6
~10–15 hrs/wk

Why this matters: Python is the main language used for AI and machine learning. Before you can teach a computer to "learn", you need to be comfortable telling it exactly what to do, step by step. This stage builds that comfort from zero.

KEY TERMS EXPLAINED

Variable	A named container that stores a piece of data, like a box with a label.
Function	A reusable block of code that performs a task when you call it.
Loop	Code that repeats an action automatically, instead of writing it many times.
Data structure	A way of organizing data — like a list (ordered items) or dictionary (key-value pairs).

WEEK 1 — Setup & First Steps

~10 hrs

- Install Python and VS Code
- Run your first Python file from the terminal
- Learn `print()`, comments, and variables
- Understand data types: `int`, `float`, `str`, `bool`
- Take user input and convert types
- Practice arithmetic, comparison, and logical operators
- Learn to read Python error messages without panicking

Build / Practice: Write a program that takes your name and age and prints a friendly message back

WEEK 2 — Decisions & Loops

~10 hrs

- `if` / `elif` / `else` statements
- Nested conditions
- `for` loops and `while` loops
- `range()` function
- `break`, `continue`, and `pass`

Build / Practice: Build a Number Guessing Game and a simple Grade Calculator

WEEK 3 — Storing Data: Lists, Dicts & More

~12 hrs

- Lists — create, access, update, sort
- Tuples and why they can't be changed
- Dictionaries — keys, values, items
- Sets and uniqueness
- Nested data structures
- String methods and slicing

Build / Practice: Build a simple To-Do List app that runs in the terminal

WEEK 4 — Functions & Working with Files

~12 hrs

- def and return
- Parameters, arguments, and default values
- *args and **kwargs (advanced, don't worry if slow)
- try / except / finally for handling errors
- Reading and writing text files
- Basics of JSON and CSV files

Build / Practice: Build a Personal Expense Tracker that saves data to a CSV file

WEEK 5 — Object-Oriented Programming (OOP)

~10 hrs

- Classes and objects
- __init__ and self
- Instance methods
- Inheritance and super()
- Encapsulation basics
- __str__ and __repr__ methods

Build / Practice: Rebuild your To-Do List app using classes instead of plain functions

WEEK 6 — Python Checkpoint Project & Git Basics

~12 hrs

- Review everything from weeks 1–5
- Clean code habits and simple debugging
- Install Git and create a GitHub account
- git init, add, commit, push, pull
- Write a clear README for a project

Build / Practice: Combine what you've learned into one small polished project on GitHub

CHECKPOINT before moving on: Can you write a 100+ line Python program — with functions and at least one class — completely from memory, without looking anything up?

PHASE 2

Python + Math Foundations

Weeks 7–12
~10–15 hrs/wk

Why this matters: Machine learning models are, underneath, made of math. You don't need to be a mathematician, but you do need to understand vectors, matrices, and a few key ideas from calculus and probability — because every explanation of "how a model learns" uses this vocabulary.

KEY TERMS EXPLAINED

Vector	A list of numbers representing a point or direction, like [3, 5, 2].
Matrix	A grid of numbers (rows and columns) — think of a spreadsheet of numbers.
Derivative	A measure of how much a value changes as its input changes slightly.
Gradient descent	A method for a model to gradually improve by taking small steps that reduce its error.

WEEK 7 — Advanced Python for Real Apps

~10 hrs

- Modules and packages
- Virtual environments (venv)
- pip and installing packages
- Environment variables and .env files
- requests library: calling an API and reading JSON
- Basic type hints

Build / Practice: Build a Public API Data Fetcher (e.g., weather or currency exchange data)

WEEK 8 — Linear Algebra for AI — Part 1

~8 hrs

- What is a scalar, vector, and matrix?
- Vector magnitude (length)
- Matrix operations: addition, subtraction
- Why AI represents data as vectors and matrices

Build / Practice: Do 10 practice problems by hand, then check them using NumPy

WEEK 9 — Linear Algebra for AI — Part 2

~8 hrs

- Matrix multiplication (step by step)
- The dot product and what it represents
- Transpose of a matrix
- Practice until multiplication "clicks" intuitively

Build / Practice: Write a small Python script that multiplies matrices without using any library

WEEK 10 — Calculus Intuition (No Heavy Math Needed)

~8 hrs

- What is a function and a derivative, in plain language
- Partial derivatives — just "derivative, but for one variable at a time"
- The chain rule, explained with simple examples
- What a gradient is

Build / Practice: Watch/read 2 beginner explanations, then explain gradient descent in your own words (out loud or in writing)

WEEK 11 — Gradient Descent From Scratch

~10 hrs

- Why models need a loss function
- How gradient descent reduces that loss step by step
- Learning rate — what happens if it's too big or too small

Build / Practice: Implement gradient descent from scratch in Python to fit a straight line to data points

WEEK 12 — Probability & Basic Statistics

~10 hrs

- Mean, median, mode
- Variance and standard deviation
- Basic probability rules
- Conditional probability
- Bayes' theorem, explained simply

Build / Practice: Solve 15 probability/statistics practice problems in a notebook

CHECKPOINT before moving on: Can you explain, in plain English to a non-programmer friend, why a model needs a loss function and how gradient descent helps it improve?

PHASE 3

Data Analysis with NumPy & Pandas

Weeks 13–16
~10–14 hrs/wk

Why this matters: Real-world data is messy — missing values, duplicates, wrong formats. Before you can train any model, you need to clean and explore data, and see what it's actually telling you. This is one of the most-used everyday skills for any AI/ML role.

KEY TERMS EXPLAINED

DataFrame	A table of data (rows and columns) that Pandas uses — like an Excel sheet inside Python.
Missing value	A gap in your data where no value was recorded.
EDA	"Exploratory Data Analysis" — the process of exploring a dataset to understand it before modeling.
Correlation	A number showing how strongly two things move together (e.g., height and weight).

WEEK 13 — NumPy Fundamentals

~10 hrs

- Why NumPy is used in AI/ML
- Arrays vs plain Python lists
- Creating arrays: zeros, ones, arange, linspace
- Indexing and slicing arrays
- Reshaping arrays
- Broadcasting (applying operations to whole arrays at once)

Build / Practice: Convert 5 of your earlier list-based mini scripts to use NumPy arrays instead

WEEK 14 — Pandas Fundamentals

~12 hrs

- Series vs DataFrame
- Loading a CSV or JSON file
- head(), info(), describe()
- Selecting rows and columns
- Filtering and sorting data
- Handling missing values (dropna, fillna)
- Removing duplicates
- GroupBy and basic aggregation

Build / Practice: Load a real dataset (e.g., students, sales) and clean it step by step

WEEK 15 — Data Visualization

~10 hrs

- Matplotlib basics
- Seaborn basics
- Line charts and bar charts
- Histograms and scatter plots
- Box plots for spotting outliers
- Correlation heatmaps

Build / Practice: Create 6 different charts explaining one dataset's story

WEEK 16 — Full EDA Project

~14 hrs

- Pick a real, free dataset
- Inspect data quality issues
- Clean missing/duplicate values
- Create meaningful visualizations
- Find patterns and correlations
- Write 5–10 clear conclusions

Build / Practice: Complete a full EDA project (e.g., student performance, sales, or movies dataset) with a written summary

CHECKPOINT before moving on: Can you take a brand-new, messy CSV file and — without help — clean it, visualize it, and write 5 honest conclusions about it?

PHASE 4

SQL for Data

Weeks 17–19
~8–10 hrs/wk

Why this matters: Most real company data lives in databases, not CSV files. SQL is the standard language for asking questions of that data ("how many users signed up last month?") — it's used constantly alongside Python in data and AI roles.

KEY TERMS EXPLAINED

Table	A structured collection of rows and columns in a database, similar to a spreadsheet.
Join	Combining rows from two tables based on a shared column (e.g., matching orders to customers).
Aggregate function	A calculation across many rows, like COUNT, SUM, or AVG.
Query	A request you write to ask a database a question or retrieve specific data.

WEEK 17 — SQL Basics

~8 hrs

- What is a relational database
- Tables, rows, columns, and keys
- SELECT, WHERE, ORDER BY
- Aggregate functions: COUNT, SUM, AVG
- GROUP BY and HAVING

Build / Practice: Set up PostgreSQL locally and run 15 basic practice queries

WEEK 18 — SQL Joins & Logic

~8 hrs

- INNER JOIN, LEFT JOIN, RIGHT JOIN
- UNION
- CASE statements (if/else inside SQL)
- Subqueries
- Handling NULL values

Build / Practice: Write queries that combine 2–3 tables to answer real questions

WEEK 19 — Advanced SQL

~10 hrs

- Common Table Expressions (CTEs)
- Window functions (running totals, rankings)
- Indexes — what they are and why they matter for speed
- Basic query optimization ideas

Build / Practice: Complete a small SQL analytics project on a multi-table sample database

CHECKPOINT before moving on: Given a new database you've never seen, can you write a query joining 2+ tables to answer a specific business question?

PHASE 5

Statistics & Probability for ML

Weeks 20–22
~8–10 hrs/wk

Why this matters: Machine learning is fundamentally applied statistics. Understanding distributions, sampling, and hypothesis testing helps you judge whether a model's results are meaningful — or just noise.

KEY TERMS EXPLAINED

Distribution	The shape/pattern of how values in a dataset are spread out (e.g., a bell curve).
Confidence interval	A range that likely contains the true value you're estimating.
P-value	A number that helps you judge if a result is likely due to chance or a real effect.
Sampling	Studying a smaller group to make conclusions about a much larger group.

WEEK 20 — Distributions & Sampling

~8 hrs

- Normal distribution and why it's everywhere
- Binomial distribution basics
- Population vs sample
- Sampling methods and bias

Build / Practice: Simulate coin flips and dice rolls in Python and plot the resulting distributions

WEEK 21 — Confidence & Hypothesis Testing

~8 hrs

- Confidence intervals, explained simply
- Null vs alternative hypothesis
- P-values — what they do and don't mean
- Common pitfalls (mistaking correlation for causation)

Build / Practice: Run a hypothesis test on a small dataset (e.g., "do two groups differ significantly?")

WEEK 22 — A/B Testing

~8 hrs

- What A/B testing is and why companies use it
- Designing a simple A/B test
- Interpreting A/B test results correctly
- Common mistakes people make with A/B tests

Build / Practice: Analyze a sample A/B test dataset and write a short recommendation based on the results

CHECKPOINT before moving on: Can you explain what a p-value actually means, and why 'correlation isn't causation', to someone with no stats background?

PHASE 6

Machine Learning Basics

Weeks 23–24
~10 hrs/wk

Why this matters: This is where everything you've learned comes together. You'll learn the core vocabulary and mental model behind every ML system, so nothing later feels like unexplained magic.

KEY TERMS EXPLAINED

Supervised learning	Training a model on data that already has the correct answers labeled.
Overfitting	When a model memorizes training data instead of learning general patterns, and performs poorly on new data.
Train/test split	Dividing data so you train on one part and honestly evaluate on a part the model has never seen.
Feature	An individual measurable input the model uses to make predictions (e.g., 'age', 'price').

WEEK 23 — Core ML Vocabulary

~6 hrs

- AI vs Machine Learning vs Deep Learning
- Traditional programming vs machine learning
- Supervised, unsupervised, and reinforcement learning
- Features, labels, and the target variable
- Train / validation / test splits

Build / Practice: Write a one-page personal glossary explaining each term in your own words

WEEK 24 — How Models Actually Learn

~8 hrs

- Overfitting vs underfitting
- The bias-variance tradeoff, explained with an analogy
- What a loss function does
- What an optimizer does
- Why we never train and test on the same data

Build / Practice: Draw (by hand or digitally) a simple diagram of the ML training process end-to-end

CHECKPOINT before moving on: Can you explain why a model can show 99% accuracy on training data but still fail badly in the real world?

PHASE 7

Regression & Classification

Weeks 25–27
~10–12 hrs/wk

Why this matters: These are the two most common types of ML problems: predicting a number (regression) or predicting a category (classification). Nearly every ML job involves one of these two patterns.

KEY TERMS EXPLAINED

Regression	Predicting a continuous number, like a house price or temperature.
Classification	Predicting a category or label, like 'spam' vs 'not spam'.
Precision/Recall	Precision: how many predicted positives were correct. Recall: how many actual positives were found.
Confusion matrix	A table showing correct vs incorrect predictions for each category.

WEEK 25 — Regression Models

~12 hrs

- Linear Regression
- Multiple Linear Regression
- Polynomial Regression
- Ridge and Lasso Regression (basic idea: preventing overfitting)
- Evaluation: MAE, MSE, RMSE, and R^2

Build / Practice: Build a House Price Prediction model using scikit-learn

WEEK 26 — Classification Models — Part 1

~12 hrs

- Logistic Regression
- K-Nearest Neighbors (KNN)
- Naive Bayes
- Decision Trees
- Accuracy, precision, recall, and F1 score

Build / Practice: Build a simple classifier (e.g., predicting pass/fail) and evaluate it properly

WEEK 27 — Classification Models — Part 2

~10 hrs

- Random Forest
- Support Vector Machines (SVM) — basic idea
- Gradient boosting — basic idea
- Confusion matrix and ROC-AUC, explained visually

Build / Practice: Compare 4 different classification models on the same dataset and pick the best one

CHECKPOINT before moving on: Given a new dataset, can you correctly decide whether it's a regression or classification problem, and justify your choice?

PHASE 8

Building & Tuning Full ML Projects

Weeks 28–31
~10–14 hrs/wk

Why this matters: Knowing algorithms isn't enough — real ML work is about building a reliable pipeline: preparing data correctly, comparing models fairly, and tuning them without cheating (data leakage). This stage turns you into someone who can ship a full ML project, not just run one model.

KEY TERMS EXPLAINED

Feature scaling	Adjusting numeric features so they're on a similar scale, which helps many models perform better.
One-hot encoding	Converting a category (like 'red', 'blue', 'green') into separate 0/1 columns a model can use.
Cross-validation	Testing a model on several different train/test splits to get a more reliable performance estimate.
Data leakage	Accidentally letting information from the test set 'leak' into training, making results look better than they really are.

WEEK 28 — Preparing Data Correctly

~10 hrs

- `train_test_split`
- Feature scaling: standardization vs normalization
- Label encoding vs one-hot encoding
- Handling missing values properly (without leakage)

Build / Practice: Prepare a raw dataset end-to-end so it's ready for modeling

WEEK 29 — Training & Comparing Models

~10 hrs

- `fit()` and `predict()`
- Comparing multiple models fairly
- Cross-validation
- Understanding and avoiding data leakage

Build / Practice: Train 3+ models on the same data and compare their cross-validated scores

WEEK 30 — Hyperparameter Tuning

~10 hrs

- Hyperparameters vs parameters, explained simply
- `GridSearchCV`
- `RandomizedSearchCV`
- Building Pipelines and using `ColumnTransformer`

Build / Practice: Tune your best model from Week 29 and measure the improvement

WEEK 31 — End-to-End ML Project

~14 hrs

- Put the full lifecycle together: EDA → preprocessing → feature engineering → train multiple models → tune → evaluate → save
- Saving a trained model to disk
- Documenting your results clearly

Build / Practice: Build a Customer Churn Prediction System from start to finish

CHECKPOINT before moving on: Can you take a brand-new dataset and, on your own, run it through the full pipeline: clean → prep → train → tune → evaluate → save?

PHASE 9

Deep Learning & PyTorch

Weeks 32–35
~12–15 hrs/wk

Why this matters: Deep learning powers image recognition, speech recognition, and today's most powerful language models. This stage demystifies neural networks by building small ones yourself, so you understand what's really happening instead of treating them as a black box.

KEY TERMS EXPLAINED

Neuron	A small computing unit in a neural network that takes inputs, applies weights, and produces an output.
Backpropagation	The process a neural network uses to figure out how to adjust its weights after making an error.
Epoch	One complete pass through the entire training dataset.
Tensor	A multi-dimensional array of numbers — PyTorch's basic building block for data.

WEEK 32 — Neural Network Fundamentals

~10 hrs

- The artificial neuron: weights and bias
- Input, hidden, and output layers
- Forward propagation
- Backpropagation, explained step by step
- Learning rate, epochs, and batch size

Build / Practice: Draw and label a simple 3-layer neural network by hand, then explain it out loud

WEEK 33 — Activations & Optimizers

~8 hrs

- ReLU, Sigmoid, Tanh, Softmax — what each does and when to use it
- MSE loss vs CrossEntropy loss
- SGD vs Adam optimizer
- Overfitting in deep learning and how to reduce it

Build / Practice: Build a notebook comparing 2 activation functions on the same small network

WEEK 34 — PyTorch Fundamentals

~14 hrs

- Installing PyTorch
- Tensors and basic tensor operations
- Autograd (automatic differentiation)
- Dataset and DataLoader
- Defining a model with `nn.Module` and `forward()`
- Writing a training loop
- Saving and loading a trained model

Build / Practice: Build and train your first neural network in PyTorch on a small dataset

WEEK 35 — Deep Learning Project

~14 hrs

- Build a digit classifier
- Build a binary classifier
- Compare training vs validation performance
- Recognize signs of overfitting during training

Build / Practice: Build a Handwritten Digit Classifier (using the MNIST dataset)

CHECKPOINT before moving on: Can you explain forward propagation and backpropagation using a simple diagram, without notes?

PHASE 10

NLP, Computer Vision & Transformers

Weeks 36–39
~10–12 hrs/wk

Why this matters: This stage bridges classic deep learning and modern AI. Transformers are the architecture behind ChatGPT, Claude, and nearly every major AI model today — understanding attention is the single most valuable concept in this entire syllabus.

KEY TERMS EXPLAINED

Tokenization	Breaking text into smaller pieces (words or sub-words) a model can process.
Embedding	Turning words or images into lists of numbers that capture their meaning.
Attention	A mechanism that lets a model decide which parts of the input are most relevant to focus on.
CNN	"Convolutional Neural Network" — a network architecture especially good at understanding images.

WEEK 36 — NLP Basics

~10 hrs

- Text preprocessing
- Tokenization
- Word embeddings (Word2Vec intuition)
- RNN/LSTM basics (older approach, good to understand)
- Attention basics — the first taste

Build / Practice: Build a Sentiment Analysis or Spam Detection model

WEEK 37 — Computer Vision Basics

~10 hrs

- Image preprocessing
- Convolutional Neural Networks (CNNs)
- Convolution and pooling, explained visually
- Transfer learning — reusing a pretrained model

Build / Practice: Build an Image Classifier using a pretrained model (transfer learning)

WEEK 38 — Transformers, Explained Simply

~10 hrs

- Why transformers replaced older architectures
- Attention and self-attention, step by step
- Query, Key, and Value — explained with an analogy
- Multi-head attention
- Encoder vs Decoder
- BERT vs GPT, in plain terms

Build / Practice: Write your own plain-English explanation of 'self-attention' — no jargon allowed

WEEK 39 — Hugging Face & Local Models

~10 hrs

- Hugging Face Hub
- Using the transformers library
- Inference pipelines (ready-made model use)
- Running models locally with Ollama
- Cloud vs local model tradeoffs

Build / Practice: Use a pretrained Hugging Face model to solve a small real task

CHECKPOINT before moving on: Can you explain a Transformer's self-attention mechanism to a friend using only everyday language, no equations?

PHASE 11

LLMs & Generative AI APIs

Weeks 40–42
~10 hrs/wk

Why this matters: Now you'll build actual applications using large language models — the same kind of technology behind tools like ChatGPT and Claude. This is often the most exciting and immediately-usable stage for beginners.

KEY TERMS EXPLAINED

Prompt	The text instruction or question you send to an AI model.
Context window	The maximum amount of text (in tokens) a model can 'see' at once.
Temperature	A setting controlling how random or predictable a model's responses are.
Function/tool calling	Letting an AI model request that your code run a specific function on its behalf.

WEEK 40 — LLM Fundamentals

~8 hrs

- What is an LLM and next-token prediction
- Pretraining and alignment (RLHF), explained simply
- Tokens and context windows
- Temperature and top-p settings
- System / user / assistant roles

Build / Practice: Manually experiment with temperature and prompts in a playground and note the differences

WEEK 41 — Building with LLM APIs

~12 hrs

- API keys and environment variables
- Sending chat requests and reading responses
- Streaming responses
- Structured (JSON) outputs
- Tool/function calling basics
- Cost and rate-limit awareness

Build / Practice: Build a simple AI Chatbot or Resume Analyzer using an LLM API

WEEK 42 — Prompt Engineering

~8 hrs

- Zero-shot vs few-shot prompting
- Writing clear, structured instructions
- Prompt templates you can reuse
- Being aware of prompt injection
- Basic ways to measure if a prompt is working well

Build / Practice: Take your Week 41 project and improve its prompts through 3 rounds of testing

CHECKPOINT before moving on: Can you build a small working chatbot from scratch, including handling errors and unexpected responses?

PHASE 12

RAG & Vector Databases

Weeks 43–44
~10 hrs/wk

Why this matters: LLMs don't know about your private documents or recent events by default. RAG (Retrieval-Augmented Generation) lets an AI look things up before answering — this is how most real "chat with your documents" products work.

KEY TERMS EXPLAINED

Embedding (vector)	A list of numbers representing the meaning of a piece of text, used for similarity search.
Vector database	A database specialized in storing and searching embeddings quickly.
Chunking	Splitting a long document into smaller pieces before turning them into embeddings.
Retrieval	The step where the system finds the most relevant pieces of text for a given question.

WEEK 43 — RAG Fundamentals

~8 hrs

- RAG vs plain prompting vs fine-tuning
- The pipeline: ingest → chunk → embed → store → retrieve → generate
- Similarity search (cosine similarity, explained simply)
- Chunking strategies and chunk overlap

Build / Practice: Manually chunk a document and calculate similarity scores between chunks by hand (small example)

WEEK 44 — Vector Databases & RAG Project

~14 hrs

- Popular vector databases: FAISS, Chroma, Qdrant
- Storing and searching embeddings
- Adding citations/sources to AI answers
- Basic evaluation: is the answer actually correct and grounded?

Build / Practice: Build an AI Document Assistant: upload a PDF, ask questions, get cited answers

CHECKPOINT before moving on: Can you explain, step-by-step, what happens between uploading a PDF and getting a cited AI answer back?

PHASE 13

AI Agents

Weeks 45–46
~10 hrs/wk

Why this matters: An AI agent can use tools, make decisions, and complete multi-step tasks on its own — like searching the web, running code, or checking a database before answering. This stage teaches you to build that responsibly, with clear limits.

KEY TERMS EXPLAINED

Agent	An AI system that can decide which tools to use and take multiple steps to complete a task.
Tool calling	Letting the AI request that a specific function or API be run, then use the result.
Agent loop	The repeating cycle of: think → act → observe result → think again, until done.
Stopping condition	A rule that tells the agent when to stop looping and give a final answer.

WEEK 45 — Agent Fundamentals

~6 hrs

- What is an AI agent
- Tool calling
- The agent loop and the ReAct pattern
- Stopping conditions
- Common failure modes: infinite loops, wrong tool choices, hallucination

Build / Practice: Design (on paper) an agent that could help you plan a trip, listing its tools and stopping conditions

WEEK 46 — Building a Simple Agent

~14 hrs

- Using LangChain/LangGraph basics
- Creating custom tools
- Handling tool errors gracefully
- Adding simple memory for context

Build / Practice: Build a small Research Agent that can search and summarize information using tools

CHECKPOINT before moving on: Can you explain why an agent might get stuck in a loop, and how you'd design a stopping condition to prevent it?

PHASE 14

Fine-Tuning & MLOps

Weeks 47–48
~10 hrs/wk

Why this matters: Sometimes prompting and RAG aren't enough — you need to actually customize a model on your own data. And once a model is live, you need to track experiments and monitor it responsibly. This is the professional, production side of AI work.

KEY TERMS EXPLAINED

Fine-tuning	Further training an existing model on your own data so it specializes in a task.
LoRA	A lightweight, efficient way to fine-tune large models without retraining everything.
Experiment tracking	Recording each model training run's settings and results so you can compare them later.
Model drift	When a deployed model's performance quietly degrades over time as real-world data changes.

WEEK 47 — Fine-Tuning Basics

~12 hrs

- When to fine-tune vs use RAG vs just prompt better
- Full fine-tuning vs LoRA/QLoRA (lightweight approaches)
- Why data quality matters more than data quantity
- Preparing an instruction dataset

Build / Practice: Fine-tune a small open model on a narrow, focused task

WEEK 48 — MLOps Basics

~8 hrs

- Experiment tracking (e.g., with MLflow)
- Model registry and versioning
- Monitoring: latency, error rates, cost
- Basic idea of model drift and why retraining is sometimes needed

Build / Practice: Set up experiment tracking for a project you built earlier in this syllabus

CHECKPOINT before moving on: Can you explain, in one paragraph, when you'd choose fine-tuning over RAG for a given problem?

PHASE 15

Deployment & System Design

Weeks 49–50
~10–12 hrs/wk

Why this matters: A model sitting in a notebook helps no one — it needs to run as a real service other people or apps can use. This stage covers the backend engineering skills that turn your AI work into something usable in the real world.

KEY TERMS EXPLAINED

API	"Application Programming Interface" — a defined way for other programs to talk to your service.
Container (Docker)	A packaged version of your app with everything it needs, so it runs the same everywhere.
Latency	How long it takes for a system to respond after a request.
Scaling	Handling more users/requests by adding more computing resources.

WEEK 49 — FastAPI Basics

~10 hrs

- Setting up a FastAPI project
- Routes and HTTP methods
- Request bodies and Pydantic models
- Async basics
- Auto-generated API docs

Build / Practice: Serve one of your earlier ML models through a simple FastAPI endpoint

WEEK 50 — Docker & Basic System Design

~12 hrs

- Writing a Dockerfile
- Docker Compose basics
- Deploying to a cloud provider (basic concepts)
- Thinking about scaling, timeouts, and fallback behavior for AI apps

Build / Practice: Containerize and deploy your FastAPI service so it's reachable on the internet

CHECKPOINT before moving on: Can you explain, step by step, what happens from a user's click to your model returning a prediction?

PHASE 16

Capstone Project & Job Preparation

Weeks 51–52
~14–18 hrs/wk

Why this matters: This is where everything comes together into one complete, portfolio-worthy project — and where you prepare to actually talk about your skills confidently in interviews.

KEY TERMS EXPLAINED

Capstone project	A larger, complete project that demonstrates multiple skills together, used to show your ability.
Portfolio	A curated collection of your best projects, shared publicly (usually on GitHub).
README	A document explaining what a project does, how to run it, and what you learned.

WEEK 51 — Build the Capstone

~30 hrs

- Plan the architecture before writing code
- Build the AI/ML backend (e.g., RAG + a simple agent)
- Add a basic frontend or interface
- Add a database if needed
- Deploy it so it's live

Build / Practice: Build and deploy a Capstone project, e.g., an AI Study Assistant (notes + document Q&A + quiz generation)

WEEK 52 — Portfolio & Interview Prep

~14 hrs

- Write clear READMEs for your best 5–7 projects
- Update your resume and LinkedIn for AI/ML roles
- Practice explaining your projects and tradeoffs out loud
- Do mock interview questions on Python, ML basics, and your projects

Build / Practice: Do at least 3 mock interviews (with a friend, mentor, or AI) covering your portfolio projects

CHECKPOINT before moving on: Can you explain your capstone project's architecture, and why you made each major decision, in under 3 minutes?

Glossary — Quick Reference

A single place to look up any term from this syllabus if you forget what it means.

Accuracy	The percentage of predictions a model got correct overall.
Agent	An AI system that can use tools and take multiple steps to complete a task.
API	A defined way for programs to communicate with each other.
Attention	A mechanism letting a model focus on the most relevant parts of its input.
Backpropagation	How a neural network learns from its errors by adjusting weights backward through the network.
Chunking	Splitting long text into smaller pieces before processing (used in RAG).
Classification	Predicting a category or label (e.g., spam vs not spam).
Confusion matrix	A table comparing predicted vs actual outcomes for each class.
Cross-validation	Testing a model on multiple data splits for a more reliable score.
Data leakage	When test information accidentally influences training, inflating results.
Embedding	Numbers representing the meaning of text, images, or other data.
Epoch	One full pass through the training dataset.
Feature	An input variable used by a model to make predictions.
Fine-tuning	Further training a pretrained model on your own, more specific data.
Gradient descent	A method for gradually reducing a model's error by small steps.
Hyperparameter	A setting you choose before training (e.g., learning rate), not learned by the model itself.
LLM	"Large Language Model" — an AI model trained on huge amounts of text to understand and generate language.
Loss function	A formula measuring how wrong a model's predictions are.
Overfitting	When a model memorizes training data instead of learning general patterns.
Prompt	The instruction or question given to an AI model.
RAG	"Retrieval-Augmented Generation" — having an AI look up information before answering.
Regression	Predicting a continuous numeric value (e.g., a price).
Tokenization	Splitting text into smaller units (tokens) a model can process.
Transformer	The neural network architecture behind modern LLMs, built around attention.
Vector database	A database built to store and search embeddings efficiently.

Final reminder: Progress isn't about finishing weeks on schedule — it's about being able to explain concepts and rebuild small projects from memory. Go back and re-do a week if a checkpoint feels hard. That's not falling behind — that's how real learning works.